

Bra-Ket Duality: Unifying Symbolic and Neural Computation Through Dirac Notation

Zhi Wang, Thomas Wang
Diraclang

Abstract

We demonstrate that symbolic computation (function invocation, pattern matching) and neural computation (transformer attention, vector operations) are not merely analogous but mathematically identical under bra-ket notation from quantum mechanics. The key insight is that both paradigms perform inner product operations to determine computational actions: symbolic systems use discrete metrics (exact name matching), while neural systems use continuous metrics (cosine similarity). We prove that symbolic tag-checking is equivalent to a single-layer neural network in the limit of zero temperature, and show that transformer attention is literally a bra-ket operator with the sandwich structure $M_1 |o\rangle \langle o| M_2 |x\rangle$, where $|o\rangle \langle o|$ represents self-projection over context tokens. A key contribution is identifying that parameters in symbolic expressions (e.g., `|action param=value>`) correspond to context mixing through vector addition in neural networks—the continuous analog of discrete parameter binding. This unification explains why large language models can successfully perform tool-calling and symbolic reasoning: they implement differentiable versions of traditional symbolic dispatch mechanisms. We introduce DIRAC, a practical programming language that seamlessly integrates both computational paradigms, demonstrating that the theoretical unification has concrete engineering applications. Our work provides a mathematical foundation for hybrid symbolic-neural architectures and addresses critiques about the limitations of pure neural approaches by showing how system-level intelligence emerges from their composition.

Keywords: bra-ket notation, symbolic computation, neural networks, transformer attention, quantum mechanics, unified computing paradigm

1 Introduction: A Simple Example

1.1 The Aristotle Syllogism

Let's start with the most famous example in logic:

All humans are mortal
Socrates is human
Therefore, Socrates is mortal

How would you implement this reasoning in code?

Traditional approach (symbolic):

```
1 # Define facts as chained rules
2 rules = {
3     'human': 'mortal',
4     'Socrates': 'human'
5 }
6
7 # Chain lookup through rules
8 def what_is(entity):
```

```

9     if entity in rules:
10         return what_is(rules[entity])    # Recursive chain
11     return entity
12
13 # Chains: Socrates -> human -> mortal
14 print(what_is('Socrates'))    # Output: mortal

```

This recursively follows the chain: “Socrates” → “human” → “mortal”, stopping when no more rules apply. It works through exact dictionary lookup.

Neural network approach: Train a model on examples like “Socrates → mortal”, “Plato → mortal”, etc. The network learns associations through continuous vector representations.

Our insight: These are the same operation, just with different “similarity metrics”!

1.2 The Bra-Ket Formulation

In quantum mechanics notation (bra-ket), we can write:

Symbolic version:

- “All humans are mortal” = $|\text{mortal}\rangle \langle \text{human}|$ (an operator)
- “Socrates is human” = $|\text{human}\rangle \langle \text{Socrates}|$ (another operator)

Chain them together:

$$|\text{mortal}\rangle \langle \text{human}| \cdot |\text{human}\rangle \langle \text{Socrates}| \cdot |\text{Socrates}\rangle \quad (1)$$

The middle parts cancel (like “human” appears in both bra and ket):

$$|\text{mortal}\rangle \underbrace{\langle \text{human} | \text{human} \rangle}_{=1} \langle \text{Socrates} | \text{Socrates} \rangle = |\text{mortal}\rangle \quad (2)$$

Neural version: Same mathematical structure, but now consider learned embeddings $|\text{human}_{\text{emb}}\rangle$ and $|\text{Socrates}_{\text{emb}}\rangle$. The inner product $\langle \text{human}_{\text{emb}} | \text{Socrates}_{\text{emb}} \rangle$ might be 0.85 (high similarity, since Socrates is human) rather than exact 1. The network computes:

$$|\text{mortal}\rangle \langle \text{human}_{\text{emb}}| \cdot |\text{human}_{\text{emb}}\rangle \langle \text{Socrates}_{\text{emb}}| \cdot |\text{Socrates}_{\text{emb}}\rangle \quad (3)$$

where each inner product is a **similarity score** instead of exact match.

1.3 Why This Matters

This simple example reveals something profound:

Symbolic reasoning = Function lookup with exact matching
Neural reasoning = Function lookup with similarity matching
Same mathematical structure!

The bra-ket notation makes this visible. Both systems:

1. Have “operators” (functions/rules) of the form $|\text{output}\rangle \langle \text{input}|$
2. Use inner products $\langle \text{pattern} | \text{query} \rangle$ to check matches
3. Chain operators to perform multi-step reasoning

The only difference:

- **Symbolic:** $\langle \text{pattern} | \text{query} \rangle \in \{0, 1\}$ (discrete, exact)
- **Neural:** $\langle \text{pattern} | \text{query} \rangle \in [0, 1]$ (continuous, fuzzy)

This explains why LLMs can do function calling and tool use—they’re implementing differentiable versions of symbolic dispatch!

1.4 The Broader Context

For decades, AI research has oscillated between two paradigms:

Symbolic AI (dominant 1950s-1980s, resurgent in modern tool-use): Logic-based reasoning, explicit rules, theorem proving—interpretable but brittle

Neural AI (explored since 1940s, dominant 1990s-present): Learned representations, gradient descent, continuous optimization—flexible but opaque

These approaches have always coexisted, with one dominating mainstream research while the other survived on the fringe. The perceptron (1958) and early neural networks existed during symbolic AI’s “golden age,” just as expert systems and logic programming continued during the neural network revival (sparked by backpropagation’s popularization in 1986).

The rise of large language models raises a deeper question: LLMs perform tasks like tool-calling and multi-step reasoning despite being “merely” neural networks trained on text. While most researchers distinguish this from traditional symbolic AI (knowledge graphs, frames, formal logic), examining the mathematical structure reveals something surprising. **Both symbolic knowledge representation and neural networks share the same underlying structure—tensor operations that can be expressed as bra-ket inner products.**

Our thesis: **Knowledge representation (whether symbolic or neural), function dispatch, and attention mechanisms all use the same mathematical operation—inner products to match patterns and select/combine results.** The traditional “symbolic vs. neural” divide is really about discrete vs. continuous similarity metrics, not fundamentally different computation.

1.5 The Bra-Ket Insight

Our central thesis: **Symbolic and neural computation share the same mathematical structure—bra-ket inner products from quantum mechanics.**

In quantum mechanics, bra-ket notation provides an elegant way to express:

- States as vectors: $|\psi\rangle$ (ket)
- Dual vectors: $\langle\phi|$ (bra)
- Inner products: $\langle\phi|\psi\rangle$ (projection/measurement)
- Operators: $|\psi\rangle\langle\phi|$ (outer product)

We show that:

- **Symbolic function lookup** = $\langle\text{def}|\text{call}\rangle$ with discrete metric
- **Neural attention** = $\sum_i |v_i\rangle\langle k_i|q_j\rangle$ with continuous metric
- **Both are inner products** differing only in metric space and selection mechanism

This is not metaphor—it’s mathematical identity.

1.6 A Concrete Example: Function Calls

Let’s see this with something familiar—calling a function:

Symbolic (Python):

```
1 def greet(name):  
2     return f"Hello, {name}!"  
3  
4 result = greet("Alice") # Output: "Hello, Alice!"
```

Bra-ket form:

- Pattern (bra): $\langle \text{greet name} |$ matches calls to “greet” with any name parameter
- Action (ket): output template that produces “Hello, [name]”
- Function call: $| \text{greet Alice} \rangle$
- Matching:

$$\langle \text{greet name} | \text{greet Alice} \rangle = 1 \text{ (exact match with binding: name} \mapsto \text{Alice)} \quad (4)$$

- Result: “Hello, Alice!” (text output from template)

Neural (Transformer Attention):

Imagine the model has seen training examples:

- “greet Alice” $\rightarrow$ “Hello, Alice!”
- “greet Bob” $\rightarrow$ “Hello, Bob!”

When given input “greet Carol”, the attention mechanism:

1. Computes similarity: query “greet Carol” vs keys [“greet Alice”, “greet Bob”]
2. Gets high scores (cosine similarity ≈ 0.9 for both)
3. Returns weighted average of values [“Hello, Alice!”, “Hello, Bob!”]
4. Produces “Hello, Carol!”

Same structure:

$$\sum_i |\text{value}_i\rangle \langle \text{key}_i | \text{query} \rangle \quad (5)$$

The transformer is doing **soft function dispatch**—looking up similar patterns instead of exact matches!

1.7 Side-by-Side Comparison

Here’s the key insight in table form:

Aspect	Symbolic (Python)	Neural (Transformer)	Bra-Ket Form
Data	Function name: “greet”	Word embedding: [0.2, -0.5, ...]	Vector $ \text{greet} \rangle$
Match	Exact string comparison	Cosine similarity	Inner product $\langle \text{key} \text{query} \rangle$
Result	0 or 1 (binary)	0.0 to 1.0 (continuous)	Real number
Selection	Single match wins	Weighted average of all	$\sum_i \text{weight}_i \cdot \text{value}_i$
Operator	Function definition	Attention weights	$ \text{value} \rangle \langle \text{key} $

Table 1: Comparison of symbolic, neural, and bra-ket representations

The punchline: Both columns use the same mathematical operation—bra-ket inner products—just with different metrics!

This explains:

- Why LLMs can call functions and use tools (soft dispatch)
- Why fine-tuning works (adjusting similarity metrics)
- Why prompting works (changing query vectors)
- Why retrieval-augmented generation works (adding more key-value pairs)

1.8 Why This Matters

Understanding this equivalence has profound implications:

1. **Theoretical:** Symbolic AI is a special case of neural networks (discrete limit)
2. **Practical:** Hybrid systems can seamlessly compose both paradigms
3. **Engineering:** We can design languages that unify symbolic and neural operations
4. **Explanatory:** This is why LLMs can use tools—they’re doing differentiable function lookup!

1.9 Contributions

This paper makes the following contributions:

1. **Intuitive examples:** Step-by-step walkthrough of Aristotle’s syllogism and function calls showing identical mathematical structure in symbolic and neural systems
2. **Mathematical unification:** Formal proof that symbolic tag-checking equals a single-layer neural network with discrete metric (Theorem 4.1)
3. **Compact attention notation:** Derivation showing transformer attention has sandwich structure $M_1 |o\rangle \langle o| M_2 |x\rangle$ where self-projection $|o\rangle \langle o|$ acts as “memory”
4. **Practical demonstration:** DIRAC programming language implementing symbolic-neural integration with bra-ket syntax
5. **Architectural framework:** Guidelines for building hybrid systems that preserve differentiability while enabling symbolic control

1.10 Paper Organization

We structure this paper to build from concrete examples to formal theory:

- **Section 1 (This section):** Simple examples showing the core insight—Aristotle’s syllogism and function calls in both symbolic and neural forms
- **Section 2:** Symbolic computation formalized—how function calls, pattern matching, and composition work in bra-ket form
- **Section 3:** Neural computation formalized—deriving the compact sandwich form $M_1 |o\rangle \langle o| M_2 |x\rangle$ for transformer attention
- **Section 4:** Core equivalence theorem—proof that symbolic = discrete limit of neural, with duality table
- **Section 5:** DIRAC language—practical demonstration of symbolic-neural integration
- **Section 6:** Related work—positioning relative to symbolic AI, neural networks, and hybrid systems
- **Section 7:** Discussion and future directions—implications, limitations, and open questions
- **Appendix A:** Mathematical preliminaries on bra-ket notation for readers unfamiliar with quantum mechanics formalism

Reading guide: If you want intuition, read Sections 1, 2, 3, and 5. If you want formal proofs, focus on Sections 3, 4, and Appendix A.

2 Symbolic Computation as Bra-Ket Operations

For readers unfamiliar with bra-ket notation, see Appendix A for mathematical background. Here we use it intuitively: $|x\rangle$ is a vector (ket), $\langle y|$ is a dual vector (bra), and $\langle y|x\rangle$ is their inner product (a number measuring similarity).

2.1 Function Invocation as Inner Product

Consider a function definition:

```
1 def greet(name):
2     return f"Hello, {name}!"
```

In traditional notation:

- **Definition signature:** `greet(name)` — the pattern to match
- **Function body:** `f"Hello, {name}!"` — the action to perform
- **Function call:** `greet("Alice")` — the invocation

Bra-ket formulation:

Define:

- $|\text{action}\rangle$ = function body (what to execute)
- $\langle \text{pattern}|$ = function signature (what to match)
- $|\text{call}\rangle$ = invocation (input)

The function lookup process:

$$\text{match} = \langle \text{pattern} | \text{call} \rangle \quad (6)$$

If match succeeds (equals 1 in discrete metric):

$$\text{result} = |\text{action}\rangle \quad (7)$$

Example: For calling `greet` with “Alice”:

- $|\text{call}\rangle = |\text{greet Alice}\rangle$
- $\langle \text{pattern}| = \langle \text{greet name}|$
- $\langle \text{pattern} | \text{call} \rangle = 1$ (exact match on function name, binding: $\text{name} \mapsto \text{Alice}$)
- Execute $|\text{action}\rangle$ (output template) $\rightarrow$ returns: “Hello, Alice!” (text output)

2.2 Multiple Function Dispatch

With multiple function definitions:

$$\text{output} = \sum_i |\text{action}_i\rangle \cdot \theta(\langle \text{pattern}_i | \text{call} \rangle - 1) \quad (8)$$

where $\theta(x)$ is the Heaviside step function: $\theta(x) = 1$ if $x \geq 0$, else 0.

Important: In real symbolic systems, at most **one** pattern matches exactly, so only one term in the sum is non-zero:

- Each $\langle \text{pattern}_i | \text{call} \rangle$ checks one function signature

- For exact matching (discrete metric), $\langle \text{pattern}_i | \text{call} \rangle \in \{0, 1\}$
- Typically only one i has $\langle \text{pattern}_i | \text{call} \rangle = 1$, others are 0
- The sum notation represents “check all definitions, return the one that matches”
- If multiple patterns match, this indicates ambiguity (usually an error in symbolic systems)

The summation form in equation (2.3) is written to **parallel the neural case** (equation 4.2), where multiple patterns can have non-zero similarity scores and contribute to a weighted output. In symbolic AI, the sum degenerates to a single term.

Discrete metric: The inner product $\langle \text{pattern} | \text{call} \rangle$ uses discrete matching:

- 1 if names and argument types match exactly
- 0 otherwise

2.3 Pattern Matching with Parameters

In DIRAC/DISH, a single pattern can match multiple inputs by binding parameters:

DIRAC definition:

```
1 <greet time|
2 |output> Good |variable name=time>
```

Bra-ket representation:

- $\langle \text{pattern} | = \langle \text{greet time} |$ (matches any call to “greet” with a time parameter)
- $| \text{action} \rangle =$ template that outputs “Good [time]”

For input $| \text{greet morning} \rangle$:

$$\langle \text{greet time} | \text{greet morning} \rangle = 1 \quad \text{with binding: } \text{time} \mapsto \text{“morning”} \quad (9)$$

Returns: “Good morning” (text output, not a ket)

For input $| \text{greet evening} \rangle$:

$$\langle \text{greet time} | \text{greet evening} \rangle = 1 \quad \text{with binding: } \text{time} \mapsto \text{“evening”} \quad (10)$$

Returns: “Good evening” (text output, not a ket)

The pattern $\langle \text{greet time} |$ matches any call of the form $| \text{greet X} \rangle$, binding the parameter to create the appropriate output. The $| \text{output} \rangle$ action produces final text, not another ket for further processing.

2.4 Chaining Subroutine Calls

When a subroutine’s body contains another ket (not $| \text{output} \rangle$), it chains to call that subroutine. This naturally arises from the bra-ket operator structure.

Example: The Aristotle syllogism from Section 1:

```
1 <Socrates |
2   |human>
3
4 <human |
5   |mortal>
6
7 <mortal |
8   |output> All things must end.
```

Calling $|\text{Socrates}\rangle$ triggers a chain:

$$|\text{mortal}\rangle \langle \text{human}| \cdot |\text{human}\rangle \langle \text{Socrates}| \cdot |\text{Socrates}\rangle \quad (11)$$

Step-by-step execution:

1. $\langle \text{Socrates} | \text{Socrates} \rangle = 1 \rightarrow$ pattern matches
2. Execute body: $|\text{human}\rangle$ (another ket, so continue chaining)
3. $\langle \text{human} | \text{human} \rangle = 1 \rightarrow$ pattern matches
4. Execute body: $|\text{mortal}\rangle$ (another ket, so continue chaining)
5. $\langle \text{mortal} | \text{mortal} \rangle = 1 \rightarrow$ pattern matches
6. Execute body: $|\text{output}\rangle \rightarrow$ produces final text: “All things must end.”

The inner products $\langle \text{human} | \text{human} \rangle$ and $\langle \text{mortal} | \text{mortal} \rangle$ both equal 1, so the operators compose seamlessly. This is **operator composition**—the mathematical structure underlying sub-routine chaining.

2.5 Symbolic Summary

Key insight: Symbolic computation uses:

- **Discrete inner products** for pattern matching
- **Exact matching** (0 or 1)
- **Winner-take-all** selection
- **Compositional operators** for chaining

Next, we show neural computation uses the same structure with continuous metrics.

3 Neural Computation as Bra-Ket Operations

3.1 Neural Networks as Weighted Sums

Consider a single-layer neural network:

$$y = \sigma(Wx + b) \quad (12)$$

where W is an $m \times n$ weight matrix, x is an n -dimensional input vector, b is an m -dimensional bias vector, and σ is activation function applied element-wise.

Bra-ket formulation:

The matrix W has rows $\langle w_i |$ for $i = 1, \dots, m$. The matrix-vector product Wx gives a vector with components:

$$(Wx)_i = \langle w_i | x \rangle \quad (13)$$

So the i -th output component is:

$$y_i = \sigma(\langle w_i | x \rangle + b_i) \quad (14)$$

The full output vector is:

$$|y\rangle = \sigma(W|x\rangle + |b\rangle) \quad (15)$$

We can also write this as a sum over basis vectors $|\alpha_i\rangle$ (the i -th standard basis vector or eigenvector of the output space):

$$|y\rangle = \sum_i y_i |\alpha_i\rangle = \sum_i \sigma(\langle w_i|x\rangle + b_i) |\alpha_i\rangle \quad (16)$$

Compare to symbolic:

$$\text{symbolic} = \sum_i |\alpha_i\rangle \cdot \theta(\langle \text{pattern}_i|x\rangle - 1) \quad (17)$$

$$\text{neural} = \sum_i |\alpha_i\rangle \cdot \sigma(\langle w_i|x\rangle + b_i) \quad (18)$$

The structures are identical! The only differences:

- Symbolic: discrete metric, step function θ (hard selection at threshold = 1)
- Neural: continuous metric, sigmoid σ (soft selection)

3.2 Transformer Attention: The Central Mechanism

Transformer attention is the cornerstone of modern AI. We now show it's literally a bra-ket operator.

3.2.1 Standard Attention Definition

For a sequence of token embeddings $\{|o_i\rangle\}_{i=1}^n$:

Projections:

- Query: $|q_j\rangle = M_q |o_j\rangle$
- Keys: $|k_i\rangle = M_k |o_i\rangle$
- Values: $|v_i\rangle = M_v |o_i\rangle$

where M_q, M_k, M_v are learned projection matrices.

Attention output:

$$\text{output}_j = \sum_i \alpha_{ij} |v_i\rangle \quad (19)$$

where attention weights:

$$\alpha_{ij} = \frac{\exp(\langle k_i|q_j\rangle / \sqrt{d})}{\sum_{i'} \exp(\langle k_{i'}|q_j\rangle / \sqrt{d})} \quad (20)$$

3.2.2 Operator Formulation

Rewrite as:

$$\text{output}_j = \sum_i |v_i\rangle \langle k_i|q_j\rangle \quad (21)$$

Factor out the query:

$$\text{output}_j = \left(\sum_i |v_i\rangle \langle k_i| \right) |q_j\rangle \quad (22)$$

Key insight: $\sum_i |v_i\rangle \langle k_i|$ is a **linear operator** built from context tokens!

This operator:

- Takes query $|q_j\rangle$ as input
- Projects onto each key direction $\langle k_i|$
- Returns weighted sum of values $|v_i\rangle$

3.2.3 Compact Sandwich Form

Now substitute the definitions of keys, values, and queries:

$$\sum_i |v_i\rangle \langle k_i| = \sum_i (M_v |o_i\rangle) (\langle o_i| M_k^T) \quad (23)$$

Factor out matrices:

$$= M_v \left(\sum_i |o_i\rangle \langle o_i| \right) M_k^T \quad (24)$$

Define the **self-projection operator**:

$$P = \sum_i |o_i\rangle \langle o_i| \quad (25)$$

This projects onto the subspace spanned by all token embeddings—it’s the “memory” of all context.

Then:

$$\sum_i |v_i\rangle \langle k_i| = M_v P M_k^T \quad (26)$$

And the full attention output:

$$\text{output}_j = M_v P M_k^T M_q |o_j\rangle \quad (27)$$

Final compact form: Define $M_1 = M_v$ and $M_2 = M_k^T M_q$:

$$\boxed{\text{output}_j = M_1 |o\rangle \langle o| M_2 |x\rangle} \quad (28)$$

where $|x\rangle = |o_j\rangle$ is the input token.

3.2.4 Interpretation of Sandwich Structure

The operator $M_1 P M_2$ has clear semantics:

1. M_2 **transforms input:** Projects $|x\rangle$ into “query space”—formulates the question
2. $P = |o\rangle \langle o|$ **performs lookup:** Self-projection over context tokens—searches memory
3. M_1 **transforms output:** Projects matched results into “answer space”—interprets response

Physical analogy:

- M_2 : Asking a question
- P : Looking up answers in a book (context)
- M_1 : Interpreting what you found

Symbolic parallel:

- DIRAC function call: Discrete lookup ($\langle \text{def} | \text{call} \rangle = 0 \text{ or } 1$)
- Transformer attention: Continuous lookup ($\langle k_i | q_j \rangle \in [0, 1]$ after softmax)
- **Same mathematical structure:** Inner product determines action

3.3 Why This Matters

The sandwich form $M_1 |o\rangle \langle o| M_2 |x\rangle$ reveals:

1. **Attention is function dispatch:** The self-projection P acts like a dictionary/database of available functions
2. **Differentiability:** Unlike symbolic systems, every component is differentiable—enabling gradient-based learning
3. **Soft selection:** Instead of winner-take-all, attention weights all matches—more robust to noise
4. **Memory mechanism:** $P = \sum_i |o_i\rangle \langle o_i|$ explicitly represents “what’s available in context”

3.4 Concrete Example: Knowledge Representation

To make this concrete, consider a classic knowledge representation problem with IS-A and HAS-A relationships.

Knowledge base:

1. **IS-A relationship:** “Zhi is Asian” $\rightarrow |Asian\rangle \langle Zhi|$
2. **HAS-A relationship:** “Asians have black hair” $\rightarrow |black\rangle \langle hair| \langle Asian|$ (a tensor with two bras)

Query: “What is Zhi’s hair color?”

Symbolic reasoning:

Apply the IS-A relationship first:

$$|Asian\rangle \langle Zhi| |Zhi\rangle = |Asian\rangle \cdot 1 = |Asian\rangle \quad (29)$$

Now apply the HAS-A relationship:

$$|black\rangle \langle hair| \langle Asian| |Asian\rangle = |black\rangle \langle hair| \cdot 1 \quad (30)$$

Finally, query with “hair”:

$$|black\rangle \langle hair| |hair\rangle = |black\rangle \cdot 1 = |black\rangle \quad (31)$$

Answer: Black

Neural analog:

In a neural network, the same relationships are represented with continuous similarities:

- IS-A: $\langle Asian \text{ embedding} | Zhi \text{ embedding} \rangle \approx 0.9$ (high similarity)
- HAS-A: $\langle hair | \langle Asian | \text{query} \rangle$ computes attention over the knowledge base

The transformer attention mechanism implements exactly this multi-hop reasoning:

1. First hop: $|Zhi\rangle$ attends to $|Asian\rangle$ (via IS-A similarity)
2. Second hop: $|Asian\rangle$ attends to $|black \text{ hair}\rangle$ (via HAS-A similarity)
3. Final output: Context-enriched $|color'\rangle$ encoding “Zhi’s hair is black”

Key insight: The tensor structure $|black\rangle \langle hair| \langle Asian|$ in symbolic KR is identical to the multi-head attention structure in transformers—both represent multi-way relationships through nested bra-ket products.

4 The Mathematical Duality: Core Theorem

4.1 Formal Equivalence Statement

Theorem 4.1 (Symbolic-Neural Equivalence). *Let $\{|\alpha_i\rangle, \langle\beta_i|\}_{i=1}^n$ be a set of action-pattern pairs (function definitions).*

Symbolic system: *For input $|x\rangle$, select $|\alpha_i\rangle$ if $\langle\beta_i|x\rangle = 1$ (exact match):*

$$\text{output}_{\text{sym}} = \sum_i |\alpha_i\rangle \cdot \theta(\langle\beta_i|x\rangle - \tau) \quad (32)$$

where $\theta(z)$ is the Heaviside step function: $\theta(z) = 1$ if $z \geq 0$, else 0, and $\tau = 1$ for exact matching.

Neural system: *For input $|x\rangle$, compute weighted sum:*

$$\text{output}_{\text{neu}} = \sum_i |\alpha_i\rangle \cdot \sigma\left(\frac{\langle\beta_i|x\rangle - \tau}{\epsilon}\right) \quad (33)$$

where σ is sigmoid function, τ is threshold, ϵ controls sharpness.

Claim:

$$\lim_{\epsilon \rightarrow 0} \text{output}_{\text{neu}} = \text{output}_{\text{sym}} \quad (34)$$

Proof. Recall sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (35)$$

Consider $\sigma(z/\epsilon)$ as $\epsilon \rightarrow 0$:

- If $z > 0$: $\sigma(z/\epsilon) \rightarrow 1$ (since $z/\epsilon \rightarrow +\infty$)
- If $z < 0$: $\sigma(z/\epsilon) \rightarrow 0$ (since $z/\epsilon \rightarrow -\infty$)
- If $z = 0$: $\sigma(0) = 0.5$

For our case, $z = \langle\beta_i|x\rangle - \tau$:

- If $\langle\beta_i|x\rangle > \tau$: $\sigma\left(\frac{\langle\beta_i|x\rangle - \tau}{\epsilon}\right) \rightarrow 1$
- If $\langle\beta_i|x\rangle < \tau$: $\sigma\left(\frac{\langle\beta_i|x\rangle - \tau}{\epsilon}\right) \rightarrow 0$

Setting τ slightly below 1 (e.g., $\tau = 0.99$), the neural system converges to:

$$\text{output}_{\text{neu}} \rightarrow \sum_i |\alpha_i\rangle \cdot \mathbb{K}(\langle\beta_i|x\rangle = 1) = \text{output}_{\text{sym}} \quad (36)$$

where $\mathbb{K}(\cdot)$ is indicator function. □

Corollary 4.2. *Symbolic tag-checking is the zero-temperature limit of a single-layer neural network.*

Key observations:

1. **Unified form:** Both equations have the same structure: $\sum_i |\alpha_i\rangle \cdot f(\langle\beta_i|x\rangle - \tau)$ where f is either step function θ or sigmoid σ .
2. **Threshold flexibility:**
 - $\tau = 1$: Rigid exact matching (symbolic case)

- $\tau < 1$ (e.g., 0.8): Approximate matching (fuzzy symbolic or neural case)
- This is exactly what DIRAC uses for tag matching and similarity-based dispatch!

3. Temperature parameter ϵ :

- $\epsilon \rightarrow 0$: Sharp boundary, $\sigma(z/\epsilon) \rightarrow \theta(z)$ (neural becomes symbolic)
- ϵ small: Approximate step function (used in practice)
- ϵ large: Smooth weighting (fully neural)

4.2 The Duality Table

The correspondence between symbolic and neural paradigms:

Aspect	Symbolic	Neural	Bra-Ket
Match	Exact (name lookup)	Similarity (cosine, dot product)	$\langle \beta x \rangle$
Metric	Discrete (0 or 1)	Continuous ($[0, 1]$)	Inner product space
Selection	Winner-take-all	Weighted average	argmax vs. softmax
Composition	Function nesting	Layer stacking	Operator product
Learning	Manual (programming)	Automatic (gradient descent)	Parameter optimization
Binding	Late (runtime lookup)	Early (learned weights)	Basis vector choice
Context	Scope/environment	Hidden state	State vector $ h\rangle$
Chaining	Call stack	Recurrence/attention	Sequential operators

Table 2: Duality between symbolic and neural computation

4.3 Implications

For AI theory:

- Symbolic AI is not “different” from neural networks—it’s a special case (discrete metric, zero temperature)
- The “symbolic vs. neural” debate is about continuous vs. discrete inner products, not fundamentally different computation

For engineering:

- We can build hybrid systems that interpolate between discrete and continuous
- Differentiability and symbolic control are compatible—just use appropriate thresholds and metrics

For understanding LLMs:

- Tool-calling works because attention implements differentiable function dispatch
- “Emergent” symbolic behavior isn’t mysterious—LLMs learn continuous approximations to discrete operations

4.4 Parameters and Context Mixing: A Key Insight

The previous sections established that symbolic and neural systems share the same bra-ket structure for pattern matching and selection. However, there’s a subtle but important question: **What is the neural correspondent of parameters in symbolic expressions?**

The parameter puzzle:

In quantum mechanics, bra-ket notation is parameter-free:

- $|\psi\rangle$: a state vector
- $\langle\phi|\psi\rangle$: inner product
- $|\phi\rangle\langle\psi|$: operator

But in symbolic AI, we need **parameterized states**:

- $|\text{set-background color=red}\rangle$
- $|\text{kick object=ball}\rangle$
- $|\text{greet person="Alice"}\rangle$

The parameter “color=red” or “object=ball” modifies the base action. This is not standard quantum notation—it’s an extension for symbolic computation.

The neural correspondent: Context mixing

In neural networks, parameters correspond to **context blending through vector addition**:

Symbolic	Neural
$ \text{action param=value}\rangle$	$ \text{action}\rangle + \alpha \cdot \text{value}\rangle$
Parameter binding (discrete)	Context mixing (continuous)
Creates parameterized state	Creates context-enriched embedding

Example: Building context

Symbolic sequence:

$$|\text{Zhi}\rangle \xrightarrow{\text{add context}} |\text{Zhi wrote}\rangle \xrightarrow{\text{add context}} |\text{Zhi wrote code}\rangle \quad (37)$$

Each step **binds a new parameter** to the growing expression.

Neural equivalent (attention):

$$|\text{code}'\rangle = \sum_i \alpha_i |v_i\rangle = \alpha_{\text{Zhi}} |\text{Zhi}\rangle + \alpha_{\text{wrote}} |\text{wrote}\rangle + \alpha_{\text{code}} |\text{code}\rangle \quad (38)$$

The weighted sum **mixes context vectors** to create an embedding that encodes “code in the context of Zhi writing.”

Why this matters:

1. **RNN context accumulation:**

$$h_t = W_h h_{t-1} + W_x x_t + b \quad (39)$$

The vector addition injects new information (x_t) into accumulated context (h_{t-1}), similar to binding a new parameter to a symbolic expression.

2. **Attention context composition:**

$$\text{output}_j = \sum_i \alpha_{ij} |v_i\rangle \quad (40)$$

The weighted sum fills in “parameters” by blending relevant context vectors.

3. Parameter-value duality:

- Symbolic: $|\text{function param}_1=v_1 \text{ param}_2=v_2\rangle$ is a discrete structure
- Neural: $|f\rangle + \alpha_1 |v_1\rangle + \alpha_2 |v_2\rangle$ is a continuous blend

Hardware instruction analogy:

This is analogous to how CPU instructions encode operations:

1	Machine instruction:	[opcode]	[arg1]	[arg2]	[arg3]
2	Example:	[ADD]	[R1]	[R2]	[R3]

In vector form:

$$|\text{instruction}\rangle = |\text{opcode}\rangle + |\text{arg}_1\rangle + |\text{arg}_2\rangle + |\text{arg}_3\rangle \quad (41)$$

Symbolic (discrete):

- Different bit positions for opcode vs arguments
- Exact values: ADD=0x01, R1=0x10, R2=0x11
- Hardware decodes by position: bits [0-7]=opcode, bits [8-15]=arg1, etc.

Neural (continuous):

- Different dimensions in embedding space for operation vs arguments
- Continuous values: embeddings in $\mathbb{R}^d$
- Network decodes by learned attention: which dimensions encode operation vs parameters

The neural network learns to **partition the embedding space** the way hardware partitions instruction bits—some dimensions encode the “opcode” (what action), others encode “arguments” (what parameters). Vector addition combines these components into a unified representation.

The insight:

Vector addition in neural networks (whether in RNNs or attention) is not arbitrary—it’s the **continuous analog of parameter binding** in symbolic systems. When we write $|\text{kick object=ball}\rangle$, the neural network approximates this by mixing the embeddings of “kick” and “ball” in a learned vector space, similar to how a CPU instruction combines opcode and argument bits.

This explains why LLMs can perform complex reasoning: they’re continuously interpolating over the space of parameterized symbolic expressions. The difference from traditional symbolic AI is that parameters are filled **fuzzily** (through vector arithmetic) rather than **discretely** (through exact binding).

5 DIRAC: Practical Symbolic-Neural Integration

5.1 Language Overview

DIRAC is a programming language that:

- Uses bra-ket syntax for symbolic operations
- Integrates neural operations (embeddings, LLM calls)
- Supports both exact and similarity-based matching
- Enables seamless composition of symbolic and neural code

Example: Simple function call

DIRAC supports both XML syntax and bra-ket shorthand:

XML syntax:

```
1 <subroutine name="greet" param-person="string:required:Person_to_greet">
2   <output>Hello, <variable name="person"/>!/</output>
3 </subroutine>
4
5 <greet person="Alice"/>
```

Bra-ket shorthand syntax:

```
1 <greet person="string"|
2   |output> Hello, |variable name="person">
```

Call (ket):

```
1 |greet person="Alice">
```

Or simplified:

```
1 |greet Alice>
```

Semantic interpretation:

The subroutine definition `<greet person="string"|` creates a pattern (bra) that can match against calls (kets).

When a call `|greet person="Alice">` is executed:

1. **Pattern matching** via inner product: $\langle \text{greet pattern} | \text{call} \rangle$ checks if the call matches the signature
2. If match succeeds (returns 1), the body executes with the bound parameters
3. The output is produced by evaluating the ket: $|\text{output}\rangle = |\text{"Hello, Alice!"}\rangle$

This is **exactly** the symbolic dispatch pattern from Section 2: match via inner product, execute when match succeeds.

5.2 Embedding-Based Tag Matching

DIRAC supports **neural tag matching** using embeddings:

```
1 <subroutine name="calculate"
2   param-expression="string:required:Mathematical_expression">
3   <!-- Tags can be used for semantic search -->
4   <eval name="result"><!-- math logic --></eval>
5   <output><variable name="result"/></output>
6 </subroutine>
```

For a call with semantic matching:

```
1 <!-- Direct call by name -->
2 <calculate expression="2+2"/>
3
4 <!-- Or use search-subroutines for semantic lookup -->
5 <search-subroutines query="arithmetic_evaluate" limit="5"/>
```

The system:

1. Computes embeddings of call tags: ["arithmetic", "evaluate"]
2. Computes embeddings of definition tags: ["math", "computation"]

3. Calculates similarity: $\langle \text{def-tags} | \text{call-tags} \rangle$ (cosine similarity)
4. If similarity $>$ threshold, executes `calculate`

This is exactly the neural dispatch we formalized!

5.3 The Aristotle Example

Classic syllogistic reasoning:

- All humans are mortal
- Socrates is human
- Therefore, Socrates is mortal

Symbolic DIRAC:

```

1 <!-- Define relationships as nested subroutines -->
2 <subroutine name="human">
3   <mortal/>
4 </subroutine>
5
6 <subroutine name="Socrates">
7   <human/>
8 </subroutine>
9
10 <!-- Chain inference: calling Socrates invokes human which invokes
    mortal -->
11 <Socrates/>
12 <!-- Output: executes mortal -->

```

Or in Dirac's bra-ket shorthand:

```

1 <human |
2   | mortal >
3
4 <Socrates |
5   | human >

```

In bra-ket form:

$$| \text{mortal} \rangle \langle \text{human} | \cdot | \text{human} \rangle \langle \text{Socrates} | \cdot | \text{Socrates} \rangle = | \text{mortal} \rangle \quad (42)$$

Neural analog (transformer attention):

- Context contains facts: “Socrates is human”, “Humans are mortal”
- Query: “Is Socrates mortal?”
- Attention mechanism:
 - High weight on “Socrates is human” (query matches “Socrates”)
 - High weight on “Humans are mortal” (intermediate result matches “human”)
 - Output: High probability for “mortal”

Same logical structure, different implementation!

5.4 LLM Integration

DIRAC can delegate to LLMs for fuzzy/neural reasoning:

```
1 <llm model="gpt-4" execute="true" feedback="true" max-iterations="5">
2   You are a helpful assistant. Analyze this code and suggest
   improvements:
3
4   <variable name="code"/>
5
6   You can use these DIRAC tags:
7   - <eval>JavaScript code</eval>
8   - <system>shell command</system>
9   - <search-subroutines query="...">
10 </llm>
```

The LLM:

- Receives context (system prompt, user code)
- Generates response using attention mechanism ($M_1 |o\rangle \langle o| M_2 |x\rangle$)
- Can invoke DIRAC actions (symbolic operations)
- Creates **ReAct loop**: reasoning + acting

This demonstrates **system-level intelligence**:

- Neural component (LLM) handles fuzzy reasoning
- Symbolic component (DIRAC) handles precise operations
- Composition creates capabilities beyond either alone

5.5 Subroutine Management

DIRAC maintains a **distributed knowledge base**:

```
1 ~/.dirac/
2 lib/
3   system/          # Built-in functions
4   user/            # User-defined
5   project/         # Project-specific
```

When calling a subroutine:

1. Check session (recently defined)
2. Search user library (saved definitions)
3. Search project library (shared with team)
4. Search system library (built-ins)

This is **lazy loading with neural search**:

- Index subroutines by name and tags
- Use embeddings for similarity matching
- Load on-demand (not all in memory)

Bra-ket view: The library is a massive operator:

$$\mathcal{L} = \sum_i |\text{subroutine}_i\rangle \langle \text{signature}_i| \quad (43)$$

Calling a subroutine = applying this operator to input.

5.6 DISH: The DIRAC Shell

DISH (DIRAC Shell) is a practical implementation of neural-symbolic integration, analogous to bash in Unix but supporting both traditional shell commands and DIRAC's bra-ket notation.

Example session:

```

1 $ dish
2 dish> ls -la                # Regular Unix command
3 dish> |greet "Alice">      # DIRAC ket (function call)
4 Hello, Alice!
5
6 dish> <calculate expr="2+2"| # Define subroutine inline
7   |eval> <variable name="expr">
8
9 dish> |calculate expr="5*3"> # Call it
10 15

```

Key innovation: Context-aware LLM integration

When DISH encounters an `<llm>` tag, it implements **automatic tool discovery**:

1. **Scan available subroutines:** Before calling the LLM, DISH searches the environment for all available DIRAC subroutines (session, user library, project library, system library)
2. **Push to context stack:** The subroutine signatures are pushed onto a context stack
3. **Augment LLM prompt:** When sending the request to the LLM, DISH attaches the available subroutines as part of the system prompt:

```

1 You have access to the following DIRAC subroutines:
2 - <calculate expr="string">: Evaluates mathematical expressions
3 - <search-file pattern="string">: Searches files for patterns
4 - <fetch-url url="string">: Fetches content from URLs
5 ...

```

4. **LLM generates DIRAC tags:** The LLM can now respond with DIRAC tags (e.g., `<calculate expr="...">`) because it knows what's available
5. **Execute returned tags:** DISH parses and executes the DIRAC tags returned by the LLM

**This is the context mechanism that unifies symbolic and neural:
Mathematically:**

- Available subroutines form a basis: $\{|\alpha_1\rangle\langle\beta_1|, |\alpha_2\rangle\langle\beta_2|, \dots\}$
- The context stack creates the self-projection operator: $P = \sum_i |\beta_i\rangle\langle\beta_i|$
- LLM attention: $M_1 P M_2 |x\rangle$ where P encodes what tools are available
- LLM output: A ket like `|calculate expr="42+17">` that matches one of the $\langle\beta_i|$ patterns

This solves the tool-calling problem:

- **Traditional approach:** Hard-code tool schemas in LLM API calls (brittle, vendor-specific)
- **DIRAC approach:** Dynamically discover available subroutines and inject into context (flexible, extensible)

The LLM's attention mechanism ($M_1 |o\rangle\langle o| M_2 |x\rangle$) naturally implements tool selection because the context $|o\rangle$ includes the tool definitions, and the query $|x\rangle$ is the user's request.

5.7 Getting Started with DIRAC

DIRAC is available as an open-source npm package. To install:

```
1 npm install -g dirac-lang
```

This installs the DIRAC interpreter and DISH shell globally. After installation, you can:

Run DIRAC files:

```
1 dirac myprogram.di
```

Launch DISH (interactive shell):

```
1 dish
```

Quick example:

```
1 dish> |greet "World">
2 Hello, World!
3
4 dish> <calculate expr |
5   |eval> <variable name="expr">
6
7 dish> |calculate expr="2+2">
8 4
```

For more information, documentation, and examples, visit: <https://diraclang.org>

6 Related Work and Positioning

6.1 Symbolic AI Heritage

Classical symbolic systems:

- Lisp (McCarthy, 1960): S-expressions, function application
- Prolog (Colmerauer, 1972): Logic programming, unification
- Expert systems (1980s): Rule-based reasoning

McCarthy’s observation on structural equivalence: McCarthy noted that Lisp S-expressions and XML share the same fundamental structure—nested, tree-based representations. Both are forms of what we’d now call “serialized abstract syntax trees.” Our work extends this: not only do Lisp and XML share structure, but this structure (nested operators acting on arguments) is mathematically equivalent to the tensor operations in neural networks when viewed through bra-ket notation.

Our contribution: Show these use discrete bra-ket inner products—same structure as neural networks.

6.2 Neural Network Theory

Universal approximation (Hornik et al., 1989):

- Neural networks can approximate any continuous function
- Our work: Shows they also approximate discrete symbolic operations (Theorem 4.1)

Attention mechanisms (Vaswani et al., 2017):

- “Attention is All You Need” introduced transformer architecture
- Our work: Derives compact sandwich form $M_1 |o\rangle \langle o| M_2 |x\rangle$ showing attention is literally bra-ket operator

6.3 Hybrid Systems

Neural-symbolic integration (Garcez et al., 2009):

- Logic Tensor Networks: Embed logic in tensor operations
- Differentiable Forth: Compile symbolic programs to differentiable form
- Our work: Show underlying unity—not “integration” but same mathematical structure

Program synthesis (Ellis et al., 2021):

- DreamCoder: Learn program abstractions from examples
- Our work: Provides theoretical foundation—programs are discrete neural networks

6.4 Quantum Computation

Quantum algorithms (Nielsen & Chuang, 2000):

- Use bra-ket notation for quantum states
- Our work: Applies same notation to classical computing, revealing hidden structure

NOT quantum computing: We use bra-ket notation as mathematical language, not quantum superposition or entanglement.

6.5 LLM Critiques and System Intelligence

LeCun’s critique (2023):

- LLMs lack grounded world models (no sensor feedback)
- Cannot do goal→plan→act (reactive, not deliberative)

Our response:

- **System-level intelligence:** Neural + symbolic + tools = complete architecture
- LLMs implement differentiable dispatch—can plan via tool composition
- Missing: Not planning capability, but grounded physical representation
- DIRAC demonstrates compositional intelligence without massive sensors

Philosophical position:

- Intelligence need not be monolithic (single model doing everything)
- Cognitive architectures compose specialized components
- Neural and symbolic are “two faces of same coin”—both use bra-ket operations

7 Discussion and Future Directions

7.1 Theoretical Implications

Unification of paradigms:

- Symbolic and neural computation are not opposites—they’re points on a continuum
- The spectrum: Discrete metric $\rightarrow$ Continuous metric
- The common structure: Bra-ket inner products

Temperature as control parameter:

- Zero temperature ($\epsilon \rightarrow 0$): Hard symbolic matching
- High temperature (ϵ large): Soft neural averaging
- Intermediate: Hybrid behavior (useful for robustness)

Compositionality:

- Both paradigms support operator composition
- Symbolic: Function nesting $g(f(x))$
- Neural: Layer stacking $L_2(L_1(x))$
- Unified: $|g\rangle \langle g| \cdot |f\rangle \langle f| \cdot |x\rangle$

7.2 Practical Applications

Improved AI architectures:

- Design networks with explicit symbolic structure
- Use attention as learnable function dispatch
- Mix discrete and continuous operations as needed

Interpretability:

- View neural networks as “soft programs”
- Attention weights = “which function got called”
- Enables debugging: “Why did model choose this action?”

Efficiency:

- Symbolic operations are deterministic (no sampling)
- Neural operations are robust (handle ambiguity)
- Hybrid systems get best of both

7.3 Limitations and Open Questions

Limitations:

1. **Scalability:** DIRAC currently limited to small-scale programs (not production-ready)
2. **Learning:** Symbolic parts require manual design (can't learn subroutine structure from data alone)
3. **Theory gaps:** We've shown equivalence in single-layer case; multi-layer composition needs deeper analysis

Open questions:

1. **Optimal temperature:** When to use discrete vs. continuous? Can this be learned?
2. **Hierarchical structure:** Can we learn symbolic abstractions (subroutines) from neural training?
3. **Verification:** Can we prove properties of hybrid systems (combining neural learning and symbolic guarantees)?
4. **Hardware:** Should we design chips that natively support bra-ket operations?

7.4 Future Research Directions

Short-term:

1. Extend Theorem 4.1 to multi-layer case
2. Prove sample complexity results (how much data needed to learn discrete structure?)
3. Benchmark DIRAC against pure symbolic and pure neural systems

Medium-term:

1. Auto-generate DIRAC subroutines from LLM interactions
2. Learn temperature/threshold parameters from data
3. Build large-scale knowledge bases using neural indexing

Long-term:

1. Theory of compositional intelligence (how do hybrid systems scale?)
2. Neurosymbolic AGI architectures (combining reasoning, learning, perception)
3. Formal verification of learned programs

7.5 Broader Impact

Education:

- Bra-ket notation unifies curricula: One formalism for symbolic AI, neural networks, quantum computing

AI Safety:

- Interpretable AI: Neural networks as “soft programs” easier to understand

- Formal verification: Symbolic components can be provably correct
- Controllability: Explicit symbolic rules prevent unintended behavior

Scientific Computing:

- Many domains use both symbolic (equations) and neural (data-driven) methods
- Unified framework enables seamless integration

8 Conclusion

We have demonstrated that symbolic and neural computation, historically viewed as opposing paradigms, are mathematically unified under bra-ket notation from quantum mechanics. The key insights:

1. **Structural identity:** Both paradigms perform inner product operations to select actions—symbolic systems use discrete metrics, neural systems use continuous metrics.
2. **Formal equivalence:** Symbolic tag-checking equals a single-layer neural network in the zero-temperature limit (Theorem 4.1).
3. **Compact attention form:** Transformer attention has sandwich structure $M_1 |o\rangle \langle o| M_2 |x\rangle$, explicitly showing self-projection over context tokens—attention is literally a bra-ket operator.
4. **Practical demonstration:** DIRAC language seamlessly integrates symbolic and neural operations, proving the theoretical unification has concrete engineering value.
5. **System-level intelligence:** Hybrid architectures (neural + symbolic + tools) achieve capabilities beyond pure neural or pure symbolic approaches, addressing critiques about LLM limitations.

This unification provides a mathematical foundation for the next generation of AI systems: architectures that fluidly combine the robustness of neural learning with the precision of symbolic reasoning. The future of AI is not “symbolic vs. neural” but rather their elegant composition, guided by the timeless mathematics of inner products.

The lesson: When you strip away implementation details—discrete vs. continuous, exact vs. approximate, hard vs. soft—you find the same mathematical skeleton: bra-ket operations determining computational actions. Symbolic and neural are two faces of the same coin, united by the language of linear algebra.

References

- [1] Dirac, P.A.M. (1939). “A New Notation for Quantum Mechanics.” *Mathematical Proceedings of the Cambridge Philosophical Society*, 35(3), 416-418.
- [2] McCarthy, J. (1960). “Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I.” *Communications of the ACM*, 3(4), 184-195.
- [3] Vaswani, A., et al. (2017). “Attention is All You Need.” *Advances in Neural Information Processing Systems*, 30.
- [4] Hornik, K., Stinchcombe, M., & White, H. (1989). “Multilayer Feedforward Networks are Universal Approximators.” *Neural Networks*, 2(5), 359-366.

- [5] Garcez, A., Broda, K., & Gabbay, D. (2009). *Neural-Symbolic Cognitive Reasoning*. Springer.
- [6] Nielsen, M.A. & Chuang, I.L. (2000). *Quantum Computation and Quantum Information*. Cambridge University Press.
- [7] Ellis, K., et al. (2021). “DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning.” *PLDI 2021*.
- [8] Colmerauer, A., et al. (1972). “Un système de communication homme-machine en français.” *Research Report, Université d’Aix-Marseille*.
- [9] LeCun, Y. (2023). “A Path Towards Autonomous Machine Intelligence.” *Technical Report*.
- [10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

A Mathematical Preliminaries on Bra-Ket Notation

This appendix provides formal definitions of bra-ket notation for readers unfamiliar with the formalism. Those comfortable with quantum mechanics notation or linear algebra can skip this section.

A.1 Bra-Ket Notation Basics

Bra-ket notation, introduced by Paul Dirac for quantum mechanics, provides a coordinate-free way to express linear algebra operations.

Kets represent vectors (column vectors in matrix notation):

$$|\psi\rangle \in \mathcal{H} \quad (44)$$

where $\mathcal{H}$ is a Hilbert space (inner product space).

Bras represent dual vectors (row vectors):

$$\langle\phi| \in \mathcal{H}^* \quad (45)$$

where $\mathcal{H}^*$ is the dual space.

Inner product (bra-ket contraction):

$$\langle\phi|\psi\rangle \in \mathbb{C} \quad (\text{or } \mathbb{R} \text{ for real vector spaces}) \quad (46)$$

Properties:

- **Linearity:** $\langle\phi|(a|\psi_1\rangle + b|\psi_2\rangle)\rangle = a\langle\phi|\psi_1\rangle + b\langle\phi|\psi_2\rangle$
- **Conjugate symmetry:** $\langle\phi|\psi\rangle = \overline{\langle\psi|\phi\rangle}$
- **Positive definite:** $\langle\psi|\psi\rangle \geq 0$, with equality iff $|\psi\rangle = 0$

Outer product creates an operator:

$$|\psi\rangle\langle\phi| : \mathcal{H} \rightarrow \mathcal{H} \quad (47)$$

Acting on a vector:

$$(|\psi\rangle\langle\phi|)|x\rangle = |\psi\rangle(\langle\phi|x\rangle) = \langle\phi|x\rangle|\psi\rangle \quad (48)$$

This operator projects $|x\rangle$ onto $\langle\phi|$ (measuring similarity) and scales $|\psi\rangle$ by that projection.

A.2 Matrix Representation

Given an orthonormal basis $\{|e_i\rangle\}_{i=1}^n$:

Ket as column vector:

$$|\psi\rangle = \sum_i \psi_i |e_i\rangle \leftrightarrow \begin{bmatrix} \psi_1 \\ \psi_2 \\ \vdots \\ \psi_n \end{bmatrix} \quad (49)$$

Bra as row vector:

$$\langle\phi| = \sum_i \phi_i^* \langle e_i| \leftrightarrow [\phi_1^* \quad \phi_2^* \quad \cdots \quad \phi_n^*] \quad (50)$$

Inner product as vector multiplication:

$$\langle\phi|\psi\rangle = \sum_i \phi_i^* \psi_i = \vec{\phi}^\dagger \vec{\psi} \quad (51)$$

Outer product as matrix:

$$|\psi\rangle \langle\phi| = \sum_{i,j} \psi_i \phi_j^* |e_i\rangle \langle e_j| \leftrightarrow \vec{\psi} \vec{\phi}^\dagger \quad (52)$$

A.3 Projection Operators

A projection operator P satisfies $P^2 = P$ (idempotent):

$$P = |\psi\rangle \langle\psi| \quad (\text{assuming } \langle\psi|\psi\rangle = 1) \quad (53)$$

For multiple orthonormal vectors:

$$P = \sum_i |\psi_i\rangle \langle\psi_i| \quad (54)$$

This projects onto the subspace spanned by $\{|\psi_i\rangle\}$.

Key property (resolution of identity): If $\{|\psi_i\rangle\}$ form a complete orthonormal basis:

$$\sum_i |\psi_i\rangle \langle\psi_i| = I \quad (55)$$

A.4 Why Bra-Ket for Computing?

Advantages over standard matrix notation:

1. **Coordinate-free:** Emphasizes operations and their meaning, not specific numerical representations
2. **Compositional:** $\langle\phi| M |\psi\rangle$ clearly shows operator M sandwiched between states
3. **Intuitive semantics:** Bra “asks a question,” ket “provides an answer,” inner product “measures match”
4. **Operator algebra:** $|\alpha\rangle \langle\beta|$ naturally represents transformations from β to α
5. **Unifying notation:** Same formalism works for symbolic (discrete) and neural (continuous) computation

B Mathematical Proofs

B.1 Projection Operator Properties

Lemma B.1. For normalized $|\psi\rangle$ with $\langle\psi|\psi\rangle = 1$, the operator $P = |\psi\rangle\langle\psi|$ satisfies $P^2 = P$ (idempotent).

Proof.

$$P^2 = (|\psi\rangle\langle\psi|)(|\psi\rangle\langle\psi|) = |\psi\rangle(\langle\psi|\psi\rangle)\langle\psi| = |\psi\rangle \cdot 1 \cdot \langle\psi| = P \quad (56)$$

□

Lemma B.2. For orthonormal basis $\{|e_i\rangle\}_{i=1}^n$, we have $\sum_i |e_i\rangle\langle e_i| = I$ (identity).

Proof. For any $|x\rangle = \sum_j x_j |e_j\rangle$:

$$\left(\sum_i |e_i\rangle\langle e_i|\right)|x\rangle = \sum_i |e_i\rangle\langle e_i|\sum_j x_j |e_j\rangle = \sum_i \sum_j x_j |e_i\rangle\delta_{ij} = \sum_i x_i |e_i\rangle = |x\rangle \quad (57)$$

□

B.2 Sigmoid Limit Behavior

Lemma B.3. For sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$\lim_{\epsilon \rightarrow 0^+} \sigma(z/\epsilon) = \begin{cases} 1 & \text{if } z > 0 \\ 1/2 & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases} \quad (58)$$

Proof. For $z > 0$: $\lim_{\epsilon \rightarrow 0^+} \frac{1}{1+e^{-z/\epsilon}} = \frac{1}{1+0} = 1$ (since $e^{-\infty} = 0$)

For $z < 0$: $\lim_{\epsilon \rightarrow 0^+} \frac{1}{1+e^{-z/\epsilon}} = \frac{1}{1+\infty} = 0$ (since $e^{+\infty} = \infty$)

For $z = 0$: $\sigma(0/\epsilon) = \sigma(0) = \frac{1}{1+1} = 1/2$ for all ϵ

□